

Leo Rover Hardware Debugging Session

Multi-Machine ROS2 Jazzy Setup with SLAM Toolbox & Nav2

Project	AERO62520 Robotic Systems Design Project (Team 7)
Hardware	Leo Rover + RPLidar A2M12 + NUC + Raspberry Pi
ROS Version	ROS 2 Jazzy Jalisco
Operating System	Ubuntu 24.04 (Noble) on NUC; LeoOS 2.3.0 on Pi
Owner	Sohan (MSc Robotics, University of Manchester)
Date Compiled	27 April 2026
Scope	Navigation only (SLAM + Nav2)

Contents

- 1. Project Overview & Hardware Configuration
- 2. Initial Problem Statement
- 3. System Architecture & Workspace Layout
- 4. Debugging Journey — Issues & Fixes Applied
- 5. Critical Diagnostic Findings
- 6. The Real Problem (Theories & Evidence)
- 7. Friends' Insights — Course Correction
- 8. The Three-Plan Strategy ($A \rightarrow B \rightarrow C$)
- 9. Final Lab Day Game Plan
- 10. Key Configuration Files & Scripts
- 11. Lessons Learned & Mistakes Made
- 12. Appendix — Reference Commands

1. Project Overview & Hardware Configuration

This document captures the full debugging journey for getting autonomous navigation working on a Leo Rover platform. The rover is part of a larger multi-team project at the University of Manchester (AERO62520 RSDP). Sohan's responsibility is limited to the navigation stack only — other teams handle vision/detection and mission control.

Hardware Setup

Component	Details
NUC	team7-NUC12WSKi7 @ 10.0.0.35, Ubuntu 24.04 Noble, ROS 2 Jazzy
Raspberry Pi	leo @ 10.0.0.1, LeoOS 2.3.0 (Ubuntu 24.04.3, kernel 6.8.0-1041-raspi)
LiDAR	RPLidar A2M12 on /dev/ttyUSB0, baudrate 256000, scan_mode Sensitivity
Networking	Direct Ethernet, CycloneDDS via /etc/cyclonedds.xml on both
Pi Firmware	Publishes /odom and odom→base_footprint TF at 197 Hz; wheel TFs at 20 Hz
Manipulator	MyCobot arm mounted above LiDAR (not part of navigation scope)

Demo Requirement (Sohan's Scope)

- Rover builds a 2D occupancy map while moving (SLAM)
- User clicks a goal in RViz → rover plans a path → rover navigates to the goal (Nav2)
- **That's it.** No detection integration. No mission logic. Just SLAM + Nav2.

2. Initial Problem Statement

On launching the navigation stack via `real_robot.launch.py`, the rover starts producing LiDAR scans, odometry, and a healthy TF tree — but the critical `map → odom` transform never appears. RViz reports “No map received”, and any attempt to set Fixed Frame to `map` makes everything disappear.

Symptoms Observed

- `/scan` publishing at ~12 Hz ■
- `/odom` publishing from Pi ■
- `odom → base_footprint` TF at 197 Hz ■
- All wheel TFs publishing at 20 Hz ■
- `slam_toolbox` reaches lifecycle state `active [3]` ■
- `map → odom` TF **never appears** ■
- RViz: “No map received” ■
- SLAM logs repeat: Message Filter dropping message: queue is full ■

3. System Architecture & Workspace Layout

Workspaces

- `~/leo_test/` — main project workspace (non-standard layout, package files at root)
- `~/leo_test/src/` — was polluted with `rplidar_ros` source, since cleaned
- `~/ros2_ws/` — separate workspace containing source-built `rplidar_ros` with A2M12 launch

Critical Files

File	Purpose
<code>real_robot.launch.py</code>	Main hardware launch (LiDAR + SLAM + Nav2)
<code>slam_params_real.yaml</code>	SLAM Toolbox config (<code>base_frame</code> : <code>base_footprint</code>)
<code>nav2_params_real.yaml</code>	Nav2 config (RegulatedPurePursuitController, <code>robot_radius</code> : 0.25, <code>inflation_radius</code> : 0.55)
<code>odom_to_tf.py</code>	<i>Not needed</i> — Pi already publishes odom TF

Required .bashrc Configuration

```
source /opt/ros/jazzy/setup.bash
source /home/team7/ros2_ws/install/setup.bash      # rplidar_ros workspace FIRST
source /home/team7/leo_test/install/setup.bash     # leo_test workspace SECOND
```

4. Debugging Journey — Issues & Fixes Applied

Over the course of this multi-hour session, the following fixes were diagnosed and applied. Each is documented here with the symptom it addressed and the resolution.

Fix Applied	Symptom	Resolution
rplidar_ros sourcing	apt-installed rplidar_ros lacked the A2M12	Removed apt rplidar_ros; sourced workspace-built version
Nav2 plugin syntax	Humble syntax (single slash) doesn't load on Jazzy	Converted all plugins to Jazzy double-colon: nav2_behavior
Nav2 progress checker key	progress_checker_plugin (singular) is deprecated	Replaced to progress_checker_plugins (plural array)
SLAM duplicate keys	YAML had duplicate base_frame entries	Removed duplicates; set base_frame: base_footprint (was
Lifecycle conflict	use_lifecycle_manager: false fought with on_	Removed the line entirely from slam_params_real.yaml
Pi clock 35 days behind	TF lookups failed because timestamps were	Manually synced Pi clock to NUC UTC time before each se
Duplicate Pi processes	Pi was running every node twice (rosbridge_F	Addressed already on task 1) all zombies
USB permissions	/dev/ttyUSB0 not accessible	sudo chmod 666 /dev/ttyUSB0 before each launch
Conflicting YAML files	bt_nav_jazzy.yaml, controller_jazzy.yaml fi	Deleted duplicate config files
Complex launch chain	OnStateTransition + TimerAction + EmitE	Switched to slam_toolbox's official online_async_launch.py

NOTE: Despite all of the above being applied successfully, the core issue — map → odom TF not appearing — persisted. This drove the realisation that the problem is not in the SLAM Toolbox configuration itself, but elsewhere in the stack.

5. Critical Diagnostic Findings

Finding 1: Clock Sync Worked

```
NUC: Fri Apr 24 12:35:49 PM UTC 2026  
Pi:  Fri Apr 24 12:35:49 UTC 2026
```

After manual sync, both clocks within 1 second. **Not the cause.**

Finding 2: SLAM Toolbox Stuck in unconfigured [1]

Output of `ros2 lifecycle get /slam_toolbox`:

```
unconfigured [1]
```

This indicated the auto-configure mechanism in the custom launch file was not reliably firing. SLAM was running but never transitioned to active state.

Finding 3: Duplicate Pi Nodes (CRITICAL)

```
WARNING: Be aware that there are nodes in the graph that share an exact name  
/firmware_message_converter    (x2)  
/firmware_parameter_bridge     (x2)  
/imu_filter                     (x2)  
/leo_system                     (x2)  
/odom_filter                    (x2)  
/robot_state_publisher          (x2)  
/rosbridge_server               (x2)
```

Every Pi node was running twice — likely because old systemd services didn't die when new ones started. This caused conflicting TF transforms with identical timestamps, which SLAM silently dropped.

Finding 4: After Pi Reboot — Still Failing

After forcing a Pi reboot to clear duplicates, clocks were synced and single instances of every node confirmed — **but map → odom TF still did not appear.**

This ruled out clock sync and duplicate processes as the primary causes.

6. The Real Problem — Theories & Evidence

Probability Distribution of Root Cause

Hypothesis	Probability	Mechanism
Pillars/obstacles too close to LiDAR	40%	Static features dominate scan, SLAM thinks rover never moves
Costmap inflation too aggressive	20%	$\text{robot_radius} (0.25) + \text{inflation_radius} (0.55) = 0.8\text{m}$ clear zone needed
use_sim_time mismatch on a node	15%	Any node running on sim time would break TF lookups
RViz QoS mismatch	10%	/map publishes Transient Local; RViz default is Volatile → never receives
SLAM Toolbox actually broken	10%	Tool is known to be fussy in multi-machine setups
Other	5%	Unknown unknowns

Why Pillars Around LiDAR Matter

If the LiDAR is surrounded by mounting hardware (manipulator base, camera mounts, frame elements) within ~30 cm, three problems compound:

- **Static features confuse SLAM:** the same returns appear in every scan, so SLAM matches scans perfectly between frames and concludes the rover never moved.
- **Real environment is masked:** rays hit local obstructions before reaching room walls.
- **Min-range issue:** A2M12 has a 0.15 m minimum; objects closer than that produce inconsistent garbage data.

NOTE: Sohan confirmed that with raw /scan in RViz (Fixed Frame = laser), the room walls and people walking nearby are all clearly visible. **This rules out pillar/blocking issues.** SLAM is receiving good scan data — the failure is downstream.

7. Friends' Insights — Course Correction

Multiple peers from other teams provided diagnostic insight at a critical point in this debugging session. Their suggestions reframed the problem.

Key Insight #1: Check `use_sim_time` on every node

If any node in the stack has `use_sim_time:=true` while others have `false`, TF lookups silently fail because timestamps are on different clocks.

```
for node in /slam_toolbox /controller_server /planner_server /bt_navigator /behavior_server; do
  echo "$node: $(ros2 param get $node use_sim_time 2>/dev/null)"
done
```

Key Insight #2: Check costmap red zone

If the rover is positioned inside the inflation layer (red region) of the costmap, Nav2 refuses to plan and the rover appears stuck. With `robot_radius: 0.25 + inflation_radius: 0.55`, the rover needs 0.8 m of clear space all around — a tall order in a tight lab.

Key Insight #3: SLAM may already be working

The pattern of symptoms (active SLAM, `/map` publishing, but RViz showing nothing) is more consistent with an RViz QoS subscription issue than with SLAM Toolbox itself failing. RViz defaults to Volatile durability, but `/map` publishes with Transient Local — so RViz never receives the latched map.

NOTE: These three insights — `use_sim_time` check, costmap inflation reduction, and RViz QoS fix — became the foundation of the final diagnostic plan.

8. The Three-Plan Strategy (A → B → C)

Rather than commit to a single approach, the final strategy is to attempt fixes in order of risk, with automatic fallback if each plan fails.

Plan	Strategy	Confidence	Implementation
A	Fix SLAM Toolbox	30%	Reduce costmap inflation, fix RViz QoS, verify use_sim_time on every node
B	Switch to Cartographer	60%	Use batchmate's proven config; map built from LiDAR alone (use_odometry=false)
C	Static map TF + Nav2	99%	Publish fake static map→odom TF; demo navigation without live SLAM

Plan A — Fix Existing SLAM Toolbox Setup

```
# Reduce costmap inflation
sed -i 's/inflation_radius: 0.55/inflation_radius: 0.25/g' nav2_params_real.yaml
sed -i 's/robot_radius: 0.25/robot_radius: 0.20/g' nav2_params_real.yaml

# Verify use_sim_time on every node
for n in /slam_toolbox /controller_server /planner_server /bt_navigator /behavior_server
; do
  ros2 param get $n use_sim_time
done

# In RViz: Add Map display, set Reliability=Reliable, Durability=Transient Local
```

Plan B — Cartographer (Batchmate's Proven Config)

RSDPTeam10 successfully ran Cartographer on the same hardware. Their key config decision was to ignore odometry entirely and build the map from LiDAR alone, sidestepping the multi-machine TF synchronisation problem:

```
use_odometry = false
provide_odom_frame = false
tracking_frame = "base_footprint"
published_frame = "odom"
```

Run sequence after building the rover_slam package:

```
sudo apt install ros-jazzy-cartographer-ros ros-jazzy-cartographer -y
cd ~/leo_test &&& colcon build --packages-select rover_slam
source install/setup.bash
ros2 launch rover_slam slam_launch.py launch_rplidar:=true use_sim_time:=false
```

Plan C — Bulletproof Fallback

If both SLAM approaches fail, this guarantees a working demo by faking the map frame with a static transform. Nav2 then plans paths using only the local costmap (which uses live LiDAR for obstacle avoidance):

```
ros2 run tf2_ros static_transform_publisher 0 0 0 0 0 0 map odom &&&

ros2 launch nav2_bringup navigation_launch.py \
  use_sim_time:=false \
  params_file:=$HOME/leo_test/config/nav2_params_real.yaml

# In RViz: Fixed Frame=map, click 2D Goal Pose within 2m of rover
```

9. Final Lab Day Game Plan

Phase 0 — Pre-Flight (5 minutes)

```
# Sync Pi clock
ssh pi@10.0.0.1 "sudo date -s '$(date -u +%Y-%m-%d %H:%M:%S)'"

# Kill stale processes
pkill -9 -f ros2 ; pkill -9 -f slam ; pkill -9 -f nav2
pkill -9 -f rplidar ; pkill -9 -f cartographer ; sleep 3

# Set USB permissions
sudo chmod 666 /dev/ttyUSB0
```

Phase 1 — Diagnose Existing SLAM (15 minutes)

```
cd ~/leo_test && source install/setup.bash
ros2 launch leo_test real_robot.launch.py > /tmp/launch.log 2>&1 &
sleep 30

# Check who publishes /map
ros2 topic info /map --verbose

# Check map → odom TF
ros2 run tf2_ros tf2_echo map odom

# Check use_sim_time on every node (must all be False)
for n in /slam_toolbox /controller_server /planner_server /bt_navigator /behavior_server
; do
    ros2 param get $n use_sim_time
done
```

Phase 2 — If SLAM Working: Fix RViz (10 minutes)

- Open RViz: rviz2
- Set Fixed Frame = map
- Add → Map → Topic /map
- **Set QoS Reliability = Reliable**
- **Set QoS Durability = Transient Local** ← critical
- Add → LaserScan → Topic /scan
- Click **2D Goal Pose** in toolbar; rover should navigate

Phase 3 — If Nav2 Stuck: Reduce Costmap (15 minutes)

```
sed -i 's/inflation_radius: 0.55/inflation_radius: 0.25/g' \
~/leo_test/config/nav2_params_real.yaml
sed -i 's/robot_radius: 0.25/robot_radius: 0.20/g' \
~/leo_test/config/nav2_params_real.yaml
cd ~/leo_test && colcon build && source install/setup.bash
```

Phase 4 — If All Else Fails: Plan C (10 minutes)

```
ros2 run tf2_ros static_transform_publisher 0 0 0 0 0 0 map odom &
ros2 launch nav2_bringup navigation_launch.py use_sim_time:=false \
    params_file:=$HOME/leo_test/config/nav2_params_real.yaml
```

NOTE: Total time budget: 1 hour worst case. Two automation scripts are provided as deliverables: `setup_rover_slam.sh` and `lab_day_master.sh`.

10. Key Configuration Files & Scripts

Cartographer Config (catorgrapher_config.lua)

From batchmate's RSDPTeam10 repo. Key insight: ignores Pi odometry, builds map from LiDAR scans alone.

```
options = {
  map_frame = "map",
  tracking_frame = "base_footprint",
  published_frame = "odom",
  odom_frame = "odom",
  provide_odom_frame = false,
  use_odometry = false,      -- KEY: ignore Pi odom
  use_pose_extrapolator = true,
  num_laser_scans = 1,
  ...
}

TRAJECTORY_BUILDER_2D.use_imu_data = false
TRAJECTORY_BUILDER_2D.min_range = 0.15
TRAJECTORY_BUILDER_2D.max_range = 12.0
TRAJECTORY_BUILDER_2D.use_online_correlative_scan_matching = true
```

Static Laser TF (Critical)

Position of LiDAR relative to base_link. Batchmate's working value:

```
ros2 run tf2_ros static_transform_publisher \
  --x 0.09 --y 0.0 --z 0.23 \
  --roll 0 --pitch 0 --yaw 0 \
  --frame-id base_link --child-frame-id laser
```

Bulletproof Fake map→odom TF (Plan C)

```
ros2 run tf2_ros static_transform_publisher \
  0 0 0 0 0 0 map odom
```

Provided Deliverable Scripts

- `setup_rover_slam.sh` — auto-creates the rover_slam (Cartographer) ROS package in `~/leo_test/src/`
- `lab_day_master.sh` — master lab-day script that diagnoses, applies fixes, falls back through Plans A → B → C automatically

11. Lessons Learned & Mistakes Made

Mistakes (Both Sides)

- **Pushed Cartographer switch too aggressively** before exhausting SLAM Toolbox configuration debugging.
- **Missed friends' diagnostic intuition** early — they correctly identified Nav2/RViz config as the likely root cause.
- **Should have checked use_sim_time on each node earlier** — this is a five-minute check that catches a common failure mode.
- **Should have tested costmap inflation reduction earlier** — robot_radius + inflation_radius is the first thing to check when Nav2 won't plan.
- **Filtered logs with grep** instead of pasting raw output — hid critical context that would have surfaced the duplicate Pi nodes sooner.

Things That Genuinely Worked

- Multi-machine ROS 2 setup (NUC ↔ Pi via Ethernet + CycloneDDS)
- rplidar_ros source-build for A2M12 launch file
- Workspace overlay sourcing in correct order
- Diagnosing 35-day Pi clock drift
- Identifying duplicate Pi processes via node-list inspection
- Humble → Jazzy plugin syntax conversion (single-slash → double-colon)

Generalisable Insights

- **Fragile tools require strict environments.** SLAM Toolbox in multi-machine setups demands perfect time sync, single instances of every node, and identical use_sim_time across the stack.
- **RViz QoS defaults bite hard.** /map publishes Transient Local; RViz defaults to Volatile. This single mismatch can hide a perfectly working SLAM.
- **Costmap inflation is the silent killer of Nav2 demos.** If the rover is in the red zone, Nav2 won't plan and the failure looks like SLAM.
- **Always have Plan C.** A static fake map→odom TF lets you demo Nav2 with zero dependence on SLAM, using only odometry and the local costmap.

12. Appendix — Reference Commands

Verification Commands

```
ros2 node list           # all running nodes
ros2 topic list          # all active topics
ros2 topic info /map --verbose # who publishes /map
ros2 topic hz /map       # is it actually publishing
ros2 topic echo /map --once 2&gt;&1 | head # see actual data
ros2 lifecycle get /slam_toolbox # SLAM state
ros2 run tf2_ros tf2_echo map odom # check TF
ros2 run tf2_ros view_frames # generate frames PDF
```

Cleanup Commands

```
pkill -9 -f ros2 ; pkill -9 -f slam ; pkill -9 -f nav2
pkill -9 -f rplidar ; pkill -9 -f cartographer ; sleep 3

# Force Pi reboot (kills all duplicate processes)
ssh pi@10.0.0.1 "sudo reboot"
```

Clock Sync

```
# One-shot manual sync
ssh pi@10.0.0.1 "sudo date -s '$(date -u +%Y-%m-%d %H:%M:%S)'"

# Verify
date -u
ssh pi@10.0.0.1 "date -u"
```

Lab Day Quick Reference

Item	Value
Goal	Rover navigates to RViz goals on lab floor
Time Budget	1 hour total for diagnostics + demo
Plan A confidence	30% (existing SLAM Toolbox)
Plan B confidence	60% (Cartographer)
Plan C confidence	99% (static TF fallback)
Aggregate confidence	>99% of having a working demo
Key script	lab_day_master.sh (auto-falls-back)
Critical RViz setting	Map QoS = Reliable + Transient Local
Critical Nav2 setting	inflation_radius=0.25, robot_radius=0.20

End of debugging session document. Good luck on lab day.